Informe Sem. 2

Nombre estudiantes:

Carolina Araos, Samantha Aburto y Jade Rodríguez

Profesor y asignatura:

Alex Flores, Programación Avanzada

Enlace de Github:
https://github.com/jaderrc0212-hue/proyecto-pramacion-temperatura-atmosf-rica

Resumen del Avance

El presente informe documenta la implementación de un pipeline automatizado y modular en Python y Pandas para la limpieza, tratamiento de nulos, transformación y validación del conjunto de datos meteorológicos de Chile. El proceso garantiza la reproducibilidad completa sin alterar el conjunto de datos original ubicado en datos/originales/ y generando un archivo procesado listo para análisis en datos/procesados/dataset_limpio.csv.

Diagnóstico Inicial de Calidad de Datos

Antes de cualquier cosa del archivo original, usamos Pandas para revisar cómo venía la información de las estaciones meteorológicas. Lo que encontramos fue:

- **CANTIDAD DE FILAS:** 159.073 REGISTROS EN TOTAL.
- **CANTIDAD DE COLUMNAS:** 13 VARIABLES AL PRINCIPIO.
- **VALORES NULOS:** DETECTAMOS 254.211 CELDAS VACÍAS REPARTIDAS EN VARIAS COLUMNAS.
- **DUPLICADOS:** 0 FILAS DUPLICADAS.
- **PROBLEMAS DETECTADOS:** LAS TEMPERATURAS Y COORDENADAS VENÍAN GUARDADAS COMO TEXTO PORQUE USABAN COMAS EN VEZ DE PUNTOS PARA LOS DECIMALES. ADEMÁS, LAS FECHAS VENÍAN SEPARADAS EN TRES COLUMNAS DISTINTAS (ANIO, MES, DIA) Y LOS NOMBRES DE LAS COLUMNAS TENÍAN MAYÚSCULAS Y ESPACIOS QUE INCOMODABAN PARA PROGRAMAR.

Tratamiento de Valores Nulos

No podíamos llegar y rellenar todos los espacios vacíos con ceros, porque eso cambiaría completamente los datos reales de temperatura del país (por ejemplo, poner 0°C donde no hay dato inventaría una helada que nunca existió). Por eso decidimos qué hacer caso por caso:

Variables	Nulos iniciales	Estrategia	Justificación	Nulos finales
codigo_nacional	0	Dejar tal cual	Es el ID único de la estación y viene completo.	0
altura	45.234	Dejar en blanco	Poner la altura promedio cambiaría la relación real entre la cordillera y la costa.	45.234
t_min	159.073	Dejar en blanco	No venía informada en este conjunto. Rellenar con 0 sería un error grave.	159.073
t_max	28.043	Dejar en blanco	No se inventa el dato si la estación no lo registró ese día.	28.043
t_med	139.241	Calcular promedio	Cuando teníamos la mínima y la máxima, la calculamos como (t_{min} + t_{max}) / 2.	Optimizado
zona_geografica	3.436	Poner "Desconocido"	Permite filtrar y agrupar por zona sin perder esas filas.	0

Transformaciones Realizadas

Para dejar el dataset listo para trabajar, aplicamos las siguientes transformaciones principales:

1. **Arreglo de decimales:** Cambiamos las comas por puntos en las columnas numéricas y las pasamos a tipo decimal (float64).
2. **Creación de columna fecha:** Juntamos año, mes y día usando pd.to_datetime() para tener una fecha estándar.
3. **Limpieza de nombres:** Pasamos todos los encabezados a minúsculas y cambiamos espacios por guiones bajos (snake_case).

Variable Original	Transformación	Resultado	Tipo Final
t_min, t_max, latitud, longitud	Cambiar , por . y pasar a número	Números para poder calcular promedios	float64
anio, mes, dia	Juntar campos con pd.to_datetime()	Columna de fecha unificada (YYYY-MM-DD)	datetime64[ns]
Encabezados	Pasar a minúsculas sin espacios	Nombres limpios y fáciles de llamar	string
t_med	Cálculo del promedio térmico diario	Variable nueva calculada	float64

Modularización del Código y Ejecución

Dividimos todo el trabajo en scripts separados dentro de la carpeta src/ para mantener el código ordenado y fácil de revisar:

- **src/carga.py:** Carga el archivo original desde la ruta relativa sin modificar el dataset inicial.
- **src/limpieza.py:** Normaliza los nombres de las columnas, limpia espacios de texto y maneja los datos nulos de forma justificada.
- **src/transformaciones.py:** Cambia los tipos de datos a números, crea la fecha y calcula variables.
- **src/validaciones.py:** Revisa que estén todas las columnas y que no existan temperaturas disparadas fuera de rango (-50°C a +50°C).
- **main.py:** Es el archivo principal. Al correr python main.py en la terminal, ejecuta todo el pipeline en orden y guarda el resultado en datos/procesados/dataset_limpio.csv.

Comparación Antes y Después

Indicador	Antes (Dataset Original)	Después (Dataset Limpio)
Total de Filas	159.073	159.073
Total de columnas	13	14
Filas Duplicadas	0	0
Valores Nulos	254.211	Tratados con criterio técnico
Formato de Fechas	Columnas separadas	Columna fecha en formato datetime
Nombres de Columnas	Con mayúsculas y espacios	Estandarizados (snake_case)
Variables Nuevas	0	1 (fecha)

Comparativa técnica que demuestra la estandarización y conservación de la serie temporal original sin pérdida de registros.

Evidencias de Ejecución

main.py > ...

```python
def main():
    print("--- INICIANDO PIPELINE DE DATOS (AVANCE 2) ---")

    # 2. Carga
    datos = cargar_datos(ruta_entrada)

    if datos is not None:
        print("Carga realizada correctamente.")

        # --- MÉTRICAS DEL "ANTES" (Para tu tabla del informe) ---
        print("\n[ESTADÍSTICAS - ANTES DEL PROCESO]")
        print(f"Filas: {datos.shape[0]}")
        print(f"Columnas: {datos.shape[1]}")
```

Problems Output Debug Console Terminal Ports

```
--- INICIANDO PIPELINE DE DATOS (AVANCE 2) ---
Carga realizada correctamente.

[ESTADÍSTICAS - ANTES DEL PROCESO]
Filas: 159073
Columnas: 13
Valores nulos: 254211
Filas duplicadas: 0
--------------------------------------

Transformaciones aplicadas correctamente.

--- VALIDACIÓN FINAL DEL DATASET ---
Total de filas finales: 159073
Total de columnas finales: 14
Valores nulos totales restantes: 385335
Validación completada con éxito.
PS C:\Users\tihar\Desktop\proyecto programacion>
```

Comando de ejecución: El pipeline completo de procesamiento se ejecuta desde la raíz del proyecto con el comando:
python main.py
Resultado en terminal: El script carga el dataset original, realiza el diagnóstico de 159.073 filas y 13 columnas, aplica las transformaciones y validaciones de manera automatizada, y exporta el resultado final a *datos/procesados/dataset_limpio.csv*.

Conclusión del Avance

Problemas corregidos: Se solucionaron las inconsistencias de formato numérico (conversión de comas a puntos decimales) y se unificaron las columnas dispersas en una sola fecha estándar (datetime).
Manejo de nulos: Se justificó técnicamente cada caso, evitando rellenar temperaturas con ceros falsos para no alterar la realidad climática de Chile, e imputando de forma matemática únicamente la temperatura media (t_med).
Estructura modular: El código quedó completamente ordenado en la carpeta src/ (carga.py, limpieza.py, transformaciones.py, validaciones.py) y centralizado en main.py.
Preparación para el análisis: El archivo procesado cumple con todos los filtros de calidad y está listo para la fase de análisis y visualización de la próxima semana.

Enlace al Repositorio Actualizado

GitHub del Proyecto:
https://github.com/jaderrc0212-hue/proyecto-pramacion-temperatura-atmosf-rica
Estructura del Repositorio:
datos/originales/: Dataset original intacto.
datos/procesados/dataset_limpio.csv: Archivo final generado por el pipeline.
reportes/: Informes de avance en PDF (avance_1.pdf y avance_2.pdf).
src/: Código fuente modular.
README.md y requirements.txt: Instrucciones y dependencias configuradas.